```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, models
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.20.0
```

```
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

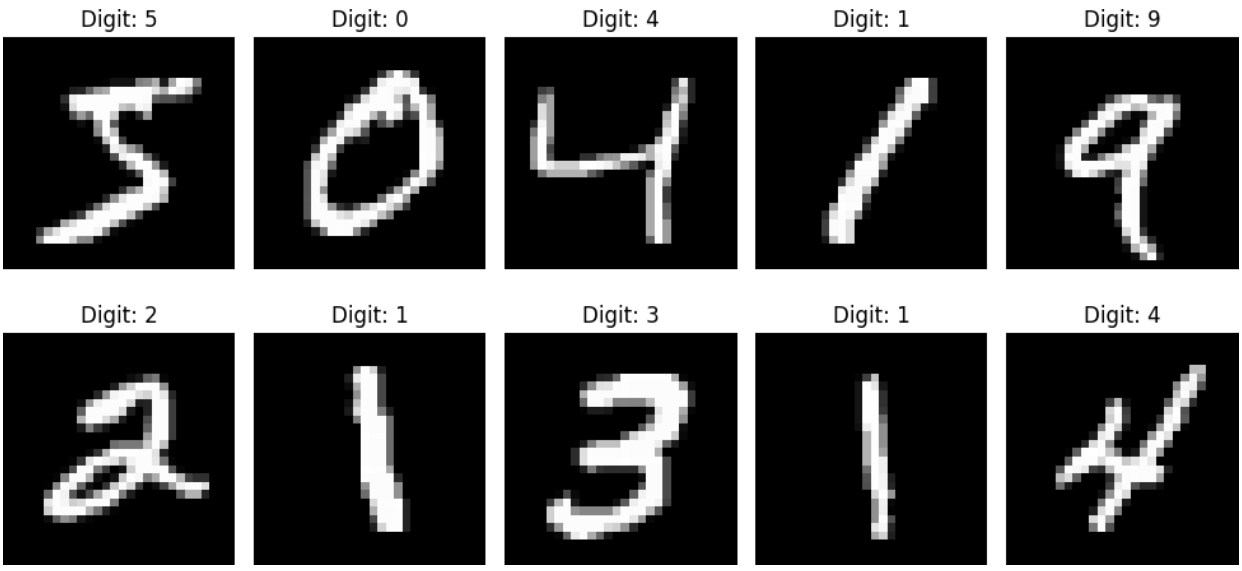
print("Training images shape:", X_train.shape)
print("Training labels shape:", y_train.shape)
print("Testing images shape:", X_test.shape)
print("Testing labels shape:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training images shape: (60000, 28, 28)
Training labels shape: (60000,)
Testing images shape: (10000, 28, 28)
Testing labels shape: (10000,)
```

```
plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[i], cmap="gray")
    plt.title(f"Digit: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# Normalize pixel values from 0-255 to 0-1
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# Add channel dimension for CNN
X_train = X_train.reshape(-1, 28, 28, 1)
```

```
X_test = X_test.reshape(-1, 28, 28, 1)

print("Training data after preprocessing:", X_train.shape)
print("Testing data after preprocessing:", X_test.shape)
```

```
Training data after preprocessing: (60000, 28, 28, 1)
Testing data after preprocessing: (10000, 28, 28, 1)
```

```
model = models.Sequential([

    layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(
        128,
        activation="relu"
    ),

    layers.Dense(
        10,
        activation="softmax"
    )
])

model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")
```

```
Model compiled successfully!
```

```
history = model.fit(
    X_train,
    y_train,
```

```
    epochs=5,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)
```

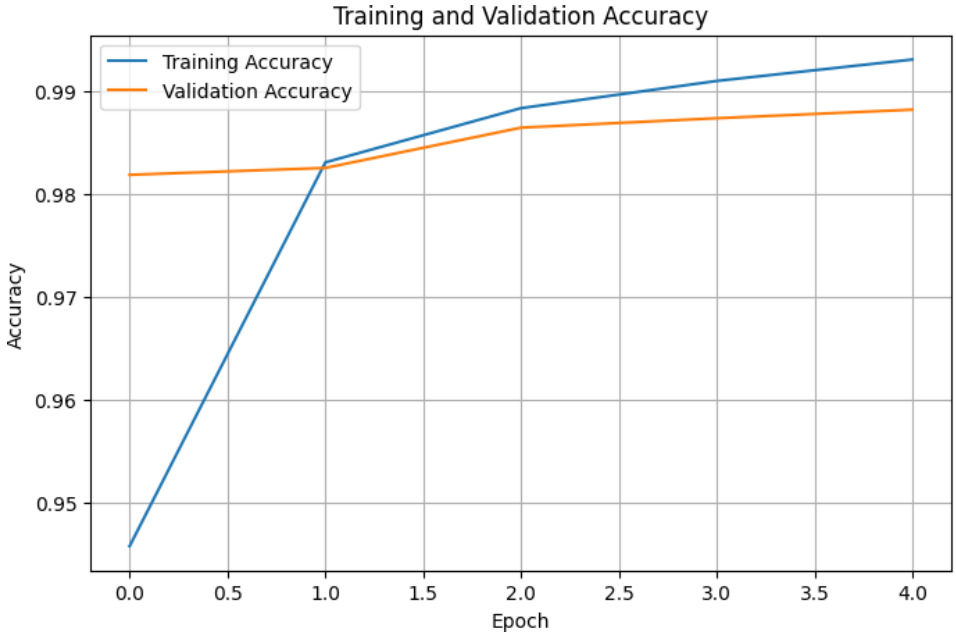
```
Epoch 1/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 58s 75ms/step - accuracy: 0.9457 - loss: 0.1820 - val_accuracy: 0.9818 - val_loss: 0.0648
Epoch 2/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 70s 93ms/step - accuracy: 0.9830 - loss: 0.0546 - val_accuracy: 0.9825 - val_loss: 0.0582
Epoch 3/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 57s 59ms/step - accuracy: 0.9883 - loss: 0.0381 - val_accuracy: 0.9864 - val_loss: 0.0422
Epoch 4/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 56ms/step - accuracy: 0.9910 - loss: 0.0279 - val_accuracy: 0.9873 - val_loss: 0.0400
Epoch 5/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 41s 55ms/step - accuracy: 0.9930 - loss: 0.0207 - val_accuracy: 0.9882 - val_loss: 0.0417
```

```
plt.figure(figsize=(8, 5))

plt.plot(history.history["accuracy"],  label="Training  Accuracy")
plt.plot(history.history["val_accuracy"],  label="Validation  Accuracy")

plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()

plt.show()
```

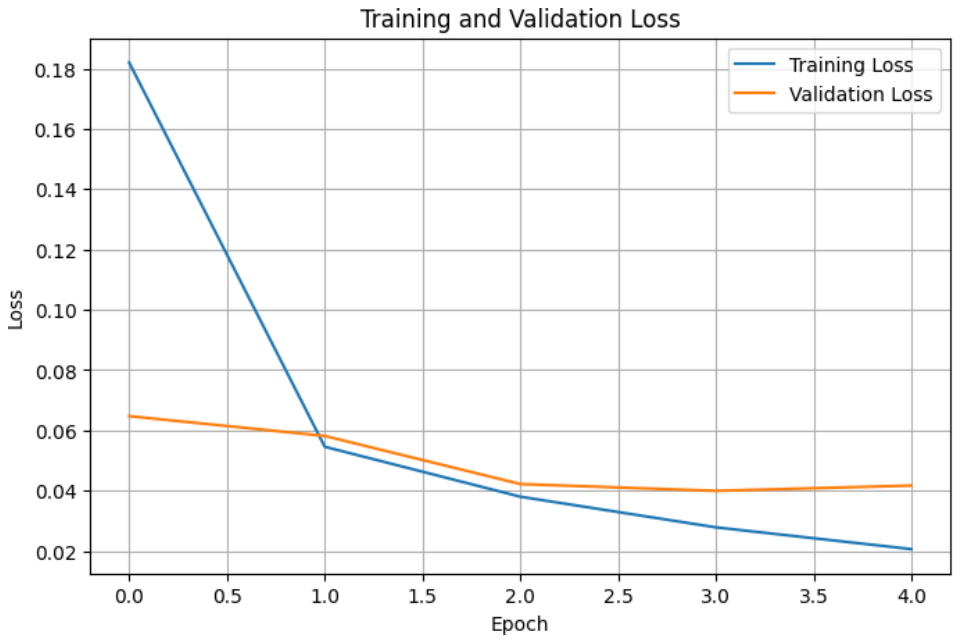



```
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"],  label="Validation  Loss")

plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid()

plt.show()
```




```
test_loss, test_accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
Test Loss: 0.02960103750228882
Test Accuracy: 0.9902999997138977
```

```
y_probability = model.predict(X_test)

y_pred = np.argmax(y_probability, axis=1)

print("First 20 Actual Labels:")
print(y_test[:20])

print("\nFirst 20 Predicted Labels:")
print(y_pred[:20])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step
First 20 Actual Labels:
[7 2 1 0 4 1 4 9 5 9 0 6 9 0 1 5 9 7 3 4]

First 20 Predicted Labels:
[7 2 1 0 4 1 4 9 5 9 0 6 9 0 1 5 9 7 3 4]
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

```
Accuracy: 0.9903
```

```
precision = precision_score(
    y_test,
    y_pred,
    average="weighted"
)

recall = recall_score(
    y_test,
    y_pred,
    average="weighted"
)

f1 = f1_score(
   y_test,
    y_pred,
    average="weighted"
)
```

```
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)
```

```
Precision: 0.9903565165211577
Recall: 0.9903
F1-Score: 0.9903016496545292
```

```
print(classification_report(y_test,  y_pred))
```

```
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       980
           1       1.00      1.00      1.00      1135
           2       0.98      1.00      0.99      1032
           3       0.99      0.99      0.99      1010
           4       1.00      0.98      0.99       982
           5       0.99      0.99      0.99       892
           6       1.00      0.98      0.99       958
           7       0.99      0.99      0.99      1028
           8       0.99      0.99      0.99       974
           9       0.98      0.99      0.98      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

```
cm = confusion_matrix(y_test, y_pred)

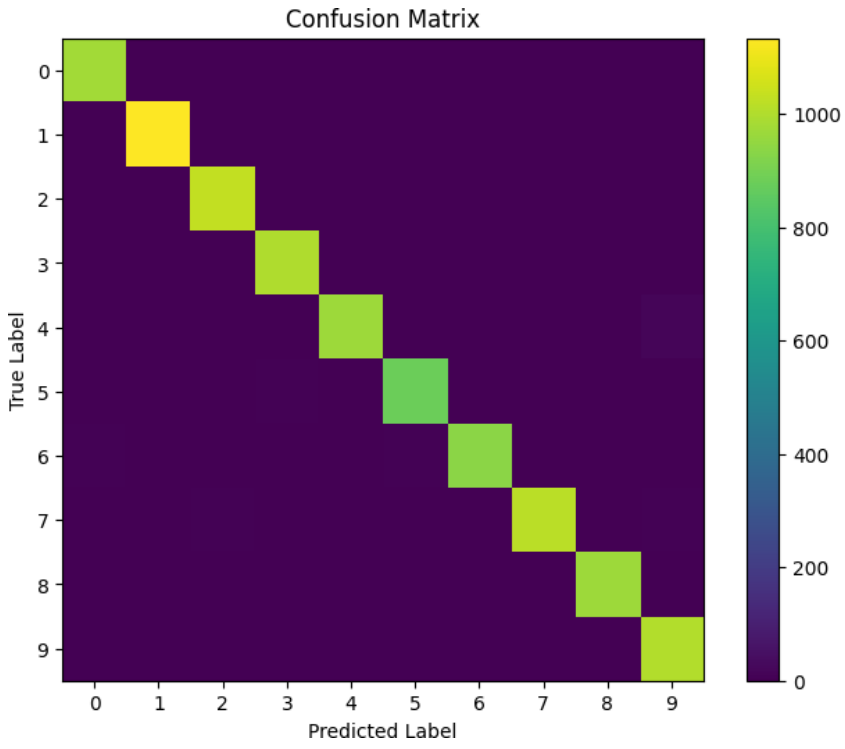
plt.figure(figsize=(8, 6))

plt.imshow(cm, interpolation="nearest")
plt.title("Confusion Matrix")
plt.colorbar()

plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.xticks(np.arange(10), np.arange(10))
plt.yticks(np.arange(10), np.arange(10))

plt.show()
```




```
plt.figure(figsize=(10, 8))

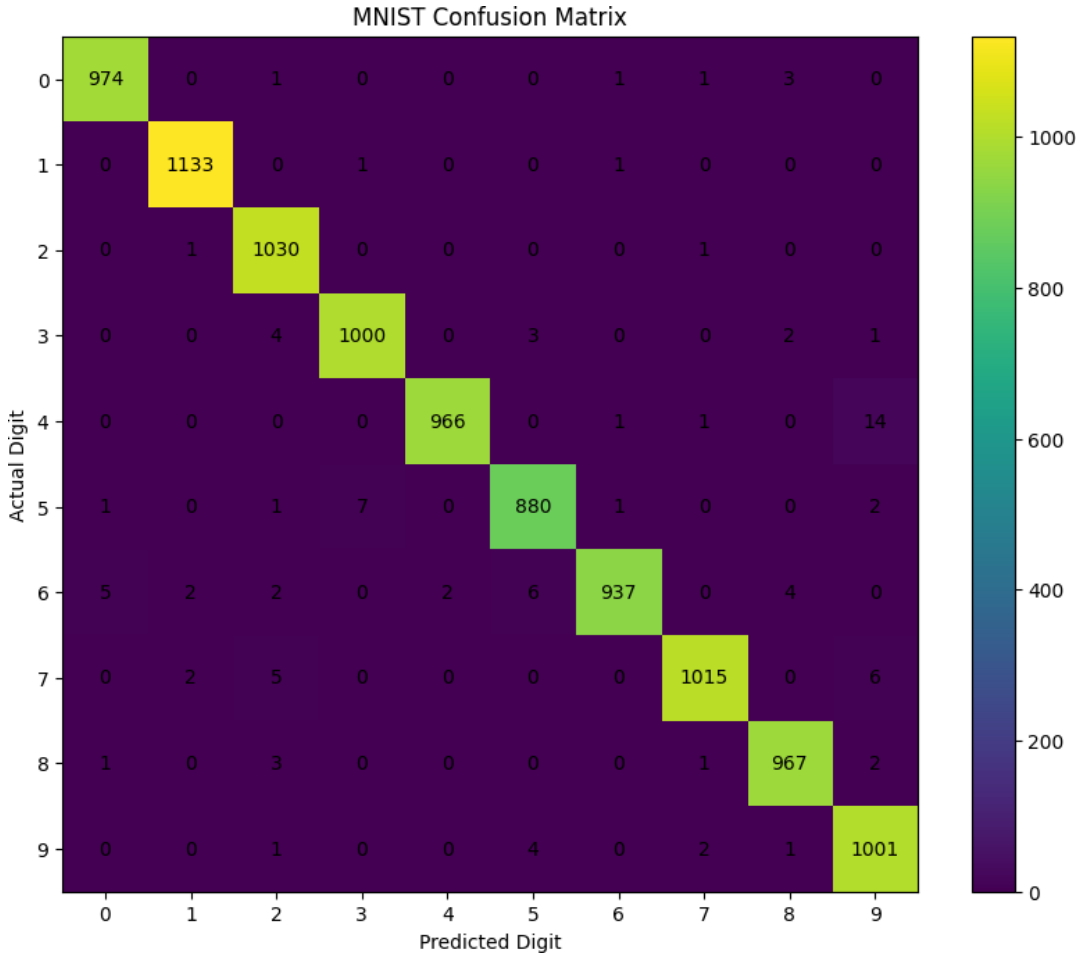
plt.imshow(cm, interpolation="nearest")
plt.title("MNIST Confusion Matrix")
plt.colorbar()
```

```
plt.xlabel("Predicted Digit")
plt.ylabel("Actual Digit")

plt.xticks(np.arange(10), np.arange(10))
plt.yticks(np.arange(10), np.arange(10))

for i in range(10):
    for j in range(10):
        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )

plt.show()
```



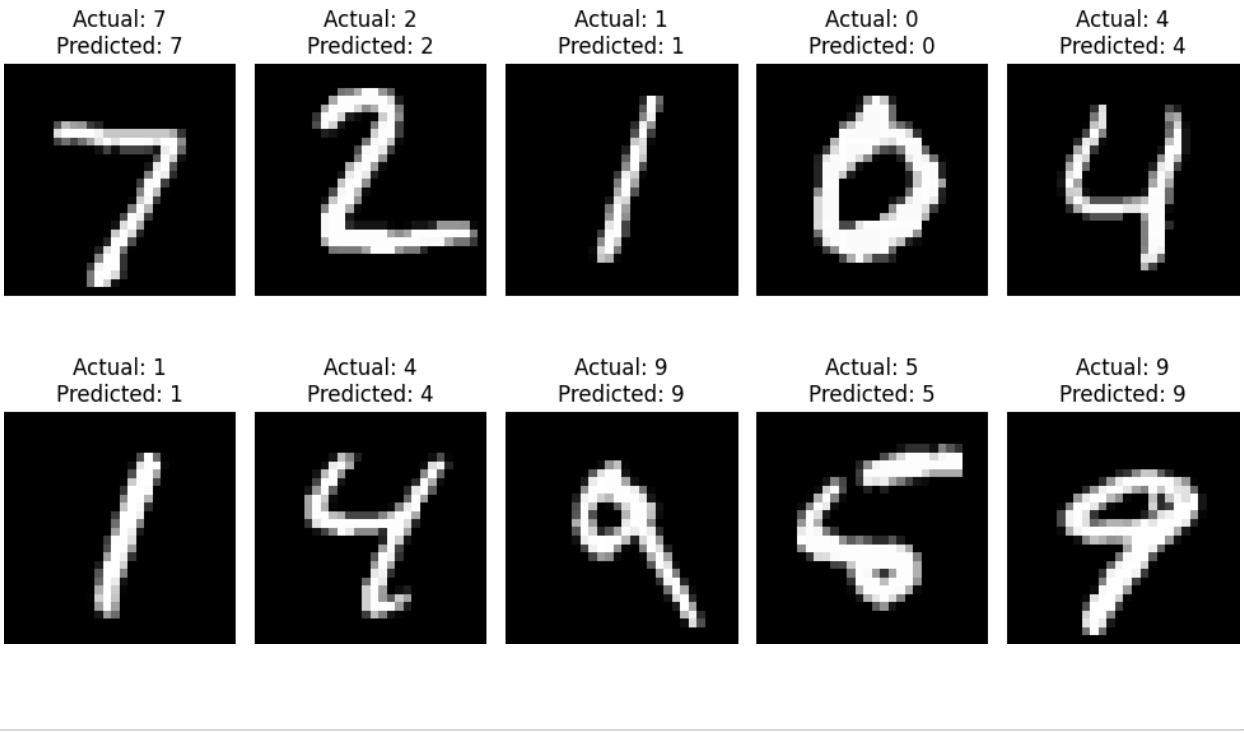

```
correct_indices = np.where(y_pred == y_test)[0]

plt.figure(figsize=(10, 6))

for i in range(10):
    index = correct_indices[i]

    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[index].reshape(28, 28), cmap="gray")
    plt.title(
        f"Actual: {y_test[index]}\nPredicted: {y_pred[index]}"
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```



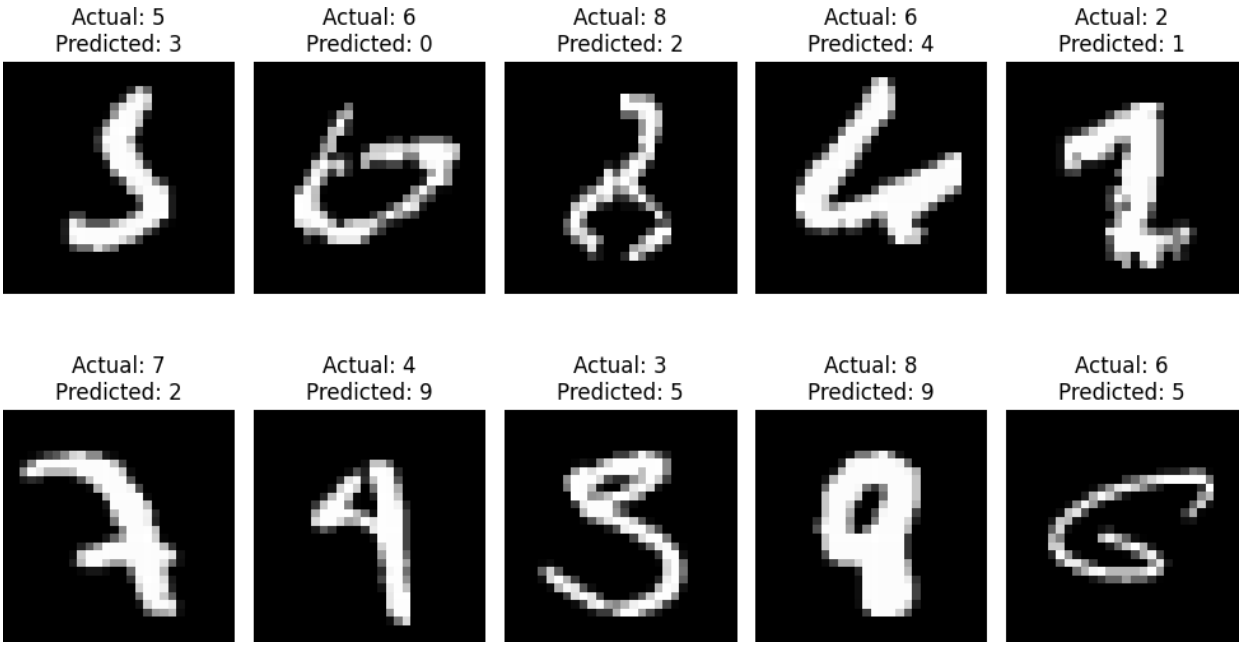

```
wrong_indices = np.where(y_pred != y_test)[0]

plt.figure(figsize=(10, 6))

for i in range(min(10, len(wrong_indices))):
    index = wrong_indices[i]

    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[index].reshape(28, 28), cmap="gray")
    plt.title(
        f"Actual: {y_test[index]}\nPredicted: {y_pred[index]}"
    )
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
index = 25

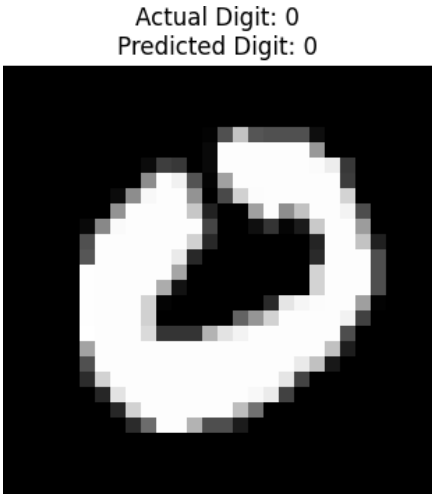
image = X_test[index]
```

```
prediction = model.predict(
    image.reshape(1, 28, 28, 1),
    verbose=0
)

predicted_digit = np.argmax(prediction)

plt.figure(figsize=(4, 4))
plt.imshow(image.reshape(28, 28), cmap="gray")
plt.title(
    f"Actual Digit: {y_test[index]}\n"
    f"Predicted Digit: {predicted_digit}"
)
plt.axis("off")
plt.show()

print("Actual Digit:", y_test[index])
print("Predicted Digit:", predicted_digit)
```




```
Actual Digit: 0
Predicted Digit: 0
```

```
confidence = np.max(prediction) * 100

print("Predicted Digit:", predicted_digit)
print("Confidence: {:.2f}%".format(confidence))
```

```
Predicted Digit: 0
Confidence: 99.99%
```

```
print("========== CNN PERFORMANCE ==========")
print(f"Test Accuracy : {accuracy:.4f}")
print(f"Precision     : {precision:.4f}")
print(f"Recall        : {recall:.4f}")
print(f"F1-Score      : {f1:.4f}")
print("=====================================")
```

```
========== CNN PERFORMANCE ==========
Test Accuracy : 0.9903
Precision     : 0.9904
Recall        : 0.9903
F1-Score      : 0.9903
=====================================
```

Ques 1. What is image classification? How does it differ from object detection and image segmentation?

Ans. Image classification is the process of assigning a class or category to an entire image based on its visual content. For example, a classification model can identify an image as a digit, character, face, or any other category. Object detection not only identifies objects but also determines their locations using bounding boxes. Image segmentation provides even more detailed information by identifying the exact pixels belonging to different objects or regions. Therefore, image classification gives an image-level class, object detection gives object-level classes along with locations, and image segmentation gives pixel-level classification.

Ques 2. Explain the architecture and working principle of a Convolutional Neural Network (CNN).

Ans. A Convolutional Neural Network, or CNN, is a deep learning model specifically designed to process and classify image data. A typical CNN consists of convolutional layers, activation functions, pooling layers, fully connected layers, and an output layer.

The convolutional layers extract important features such as edges, textures, and shapes from the image. Pooling layers reduce the spatial dimensions of the feature maps while retaining important information. The extracted features are then passed to fully connected layers for classification, and the output layer produces the final class prediction. CNNs automatically learn useful hierarchical image features during training, which reduces the need for manually designed feature extraction techniques.

Ques 3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN?

Ans. Convolutional layers are responsible for extracting important features from an image by applying learnable filters to the input. In the initial layers, they may detect simple features such as edges and corners, while deeper layers can learn more complex paflerns and shapes. Pooling layers reduce the spatial dimensions of feature maps, which decreases computational requirements and helps retain important features. Common pooling methods include max pooling and average pooling. Fully connected layers receive the extracted features and use them to perform the final classification. Thus, convolutional layers perform feature extraction, pooling layers perform dimensionality reduction, and fully connected layers perform classification.

Ques 4. Why is image normalization performed before training a deep learning model?

Ans. Image normalization is performed to scale the pixel values of images to a suitable range before training a deep learning model. For example, pixel values that originally range from 0 to 255 can be converted to a range such as 0 to 1 by dividing them by 255. Normalization helps the neural network train more efficiently because the input values become smaller and more consistent. It can improve numerical stability and help optimization algorithms converge faster. Therefore, normalization is an important preprocessing step before training CNN-based image classification models.

Ques 5. Explain the purpose of activation functions such as ReLU and Softmax in CNNs.

Ans. Activation functions introduce non-linearity into a neural network, allowing it to learn complex paflerns. ReLU, or Rectified Linear Unit, is commonly used in convolutional and hidden layers. It outputs zero for negative values and keeps positive values unchanged, which helps the network learn non-linear relationships efficiently. Softmax is generally used in the output layer of a multi-class classification network. It converts the output values into probabilities whose total sum is one. For example, if a model classifies digits from 0 to 9, Softmax can provide the probability associated with each digit. Therefore, ReLU mainly supports feature learning in hidden layers, while Softmax is used to produce class probabilities at the output.

Ques 6. What is a Confusion Matrix? How is it used to evaluate classification performance?

Ans. A Confusion Matrix is a table used to evaluate the performance of a classification model by comparing the actual class labels with the predicted class labels. It shows how many samples were correctly or incorrectly classified for each class. Correct predictions appear along the main diagonal of the matrix, while incorrect predictions appear in the other cells. A confusion matrix helps identify which classes are being predicted correctly and which classes are commonly confused with each other. It can also be used to derive evaluation measures such as accuracy, precision, recall, and F1-score.

Ques 7. Differentiate between Accuracy, Precision, Recall, and F1-Score with suitable examples.

Ans. Accuracy measures the proportion of total predictions that are correct. Precision measures how many of the samples predicted as a particular class are actually members of that class. Recall measures how many of the actual samples belonging to a class were correctly identified by the model. F1-Score is the harmonic mean of precision and recall and provides a combined measure of both. For example, if a model identifies images as containing a particular character, precision tells us how many of its positive predictions are correct, while recall tells us how many of the actual character images were successfully detected. Accuracy considers all predictions, whereas precision, recall, and F1-score provide more detailed information about classification performance.

Ques 8. Compare traditional feature-based image classification methods such as SIFT/HOG + SVM with CNN-based classification.

Ans. Traditional feature-based image classification methods first use manually designed or engineered features such as SIFT or HOG and then use a classifier such as SVM for classification. These methods depend on selecting suitable features and designing an appropriate processing pipeline. CNN-based classification learns useful features automatically from the training images through convolutional layers and does not require the same level of manual feature engineering. CNNs can learn hierarchical representations ranging from simple edges to complex shapes and paflerns. Traditional methods may require less computational power for smaller tasks, while CNNs generally provide a more automatic and scalable approach for complex image classification when sufficient training data and computational resources are available.

Ques 9. Mention five real-world applications of CNN-based image classification.

Ans. CNN-based image classification has many real-world applications. It is used in handwriflen digit recognition to identify numbers in images and documents. It is also used in facial recognition systems to classify or identify faces. In biometric authentication, CNNs can assist in recognizing individuals based on biometric images. In medical diagnosis, CNNs can classify medical images to assist in identifying relevant visual paflerns. CNN-based classification is also used in intelligent document processing to recognize characters, symbols, and document content. These applications demonstrate the importance of CNNs in modern computer vision systems.

Ques 10. How can techniques such as Data Augmentation, Transfer Learning, and Hyperparameter Tuning improve the performance of deep learning-based image classification models?

Ans. Data Augmentation improves model performance by generating modified versions of training images using operations such as rotation, flipping, cropping, scaling, or other transformations. This increases the diversity of training data and can help reduce overfifling. Transfer Learning uses a model that has already been trained on a large dataset and adapts it to a new classification task. This can reduce training requirements and improve performance, especially when the new dataset is relatively small. Hyperparameter Tuning involves selecting suitable values for parameters such as learning rate, batch size, number of epochs, and network architecture. Proper tuning can improve both the training process and final classification performance. Together, these techniques can improve the accuracy, robustness, and generalization ability of CNN-based image classification models.